VHDL 실습 과제 2 보고서

디지털논리회로설계 · VHDL_1 Problems 1~5 · 2026-10-29

1. 개요

강의자료 VHDL_1의 1.4 Problems에 있는 다섯 문제를 VHDL로 작성하였다. 각 설계는 Questa 테스트벤치로 먼저 검증한 뒤, Logisim-evolution 회로에 VHDL 엔티티로 넣어 시뮬레이션하였다(Simulate → VHDL Simulation Enabled).

문제	내용	VHDL	Logisim
1	4입력 AND / OR	p1_logic.vhd	p1_logic.circ
2	8-to-1 멀티플렉서	p2_mux8_1.vhd	p2_mux8_1.circ
3	7세그먼트에 문자 A b C d F g 표시	p3_led_letter.vhd	p3_led_letter.circ
4	25인 투표기	p4_vote25.vhd	p4_vote25.circ
5	13비트 입력의 1 개수를 7세그먼트에 표시	p5_count_ones.vhd	p5_count_ones.circ

테스트벤치는 `tb/` 폴더에 있고, 실행 결과 전체는 `questa_log.txt`에 있다. 7세그먼트 출력 `display(7 downto 0)`의 비트 순서는 강의자료의 `led` 예제와 같이 a b c d e f g dp이다.

2. Problem 1 — 4입력 AND / OR

2.1 설계

`out1 <= a and b and c and d;`는 네 입력이 모두 1일 때만 1이고, `out2 <= a or b or c or d;`는 입력 중 하나라도 1이면 1이다.

```
-- Problem 1: 4-input AND / OR
library ieee;
use ieee.std_logic_1164.all;

entity logic_test is
  port (
    a, b, c, d : in  std_logic;
    out1       : out std_logic;
    out2       : out std_logic
  );
end logic_test;

architecture bhv of logic_test is
begin
  out1 <= a and b and c and d;
  out2 <= a or b or c or d;
end bhv;
```

- **entity logic_test**: 1비트 입력 a, b, c, d와 1비트 출력 out1, out2를 정의한다.
- **architecture bhv**: 두 출력을 동시 신호 대입문(`<=`)으로 기술하였다. 두 문장은 순서와 관계없이 동시에 동작한다.

2.2 Logisim 시뮬레이션

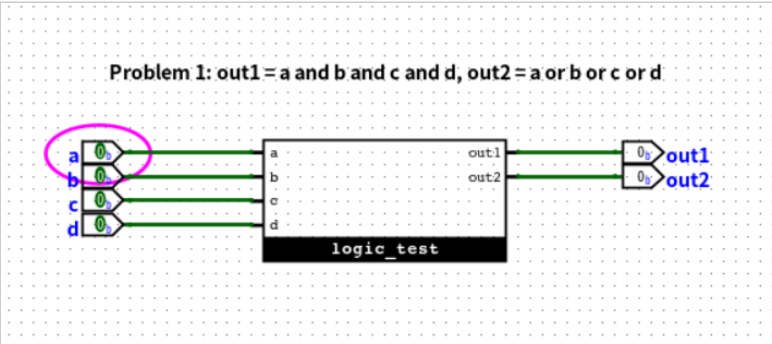


그림 1. abcd = 0000 → out1 = 0, out2 = 0

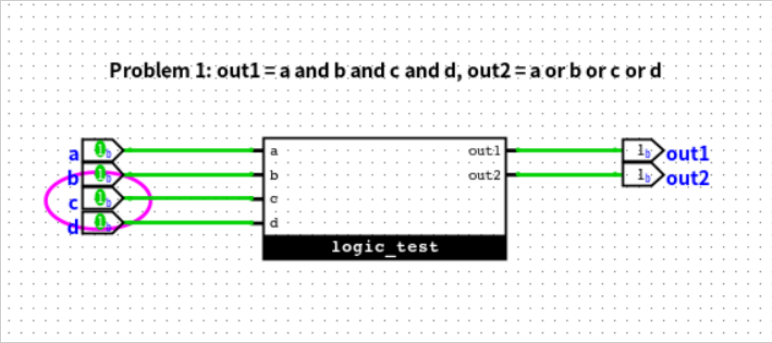


그림 2. abcd = 1111 → out1 = 1, out2 = 1

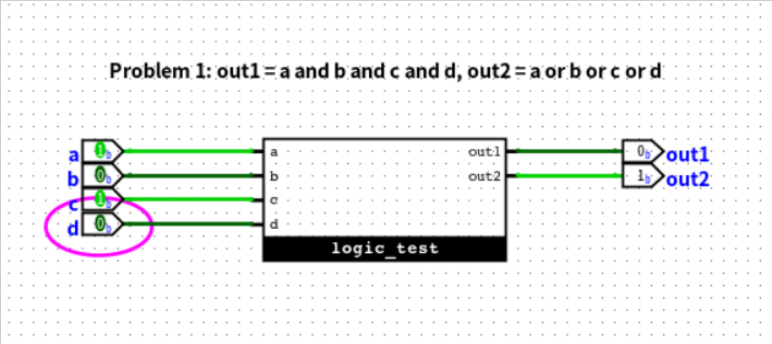


그림 3. abcd = 1010 → out1 = 0, out2 = 1

a b c d	기대 out1	기대 out2	Logisim out1	Logisim out2	판정
0 0 0 0	0	0	0	0	일치
1 1 1 1	1	1	1	1	일치
1 0 1 0	0	1	0	1	일치

Questa 테스트벤치에서는 16가지 입력 조합을 모두 넣어 out1은 1111에서만, out2는 0000에서만 값이 다른 것을 확인하였다.

```
# abcd=0000 -> out1='0' out2='0'
# abcd=0001 -> out1='0' out2='1'
# abcd=1110 -> out1='0' out2='1'
# abcd=1111 -> out1='1' out2='1'
# Errors: 0, Warnings: 0
```

3. Problem 2 — 8-to-1 멀티플렉서

3.1 설계

강의의 4-to-1 MUX(`case ch is`)를 확장하여 데이터 입력 8개(a~h)와 3비트 선택 신호 `ch`를 사용하였다.

```
-- Problem 2: 8-to-1 multiplexer
library ieee;
use ieee.std_logic_1164.all;

entity mux8_1 is
  port (
    a, b, c, d, e, f, g, h : in  std_logic;
    ch                     : in  std_logic_vector(2 downto 0);
    data                   : out std_logic
  );
end mux8_1;

architecture bhv of mux8_1 is
begin
  process (a, b, c, d, e, f, g, h, ch)
  begin
    case ch is
      when "000"  => data <= a;
      when "001"  => data <= b;
      when "010"  => data <= c;
      when "011"  => data <= d;
      when "100"  => data <= e;
      when "101"  => data <= f;
      when "110"  => data <= g;
      when "111"  => data <= h;
      when others => data <= '0';
    end case;
  end process;
end bhv;
```

- **ch : std_logic_vector(2 downto 0)**: 입력이 8개이므로 선택 신호는 3비트($2^3 = 8$)이다.
- **process (a, …, h, ch)**: 감시 목록의 신호가 바뀔 때마다 process가 다시 실행된다.
- **case ch is**: ch 값에 따라 해당 입력을 data에 대입한다. `when others`는 'X', 'Z' 같은 나머지 std_logic 값을 처리한다.

ch	000	001	010	011	100	101	110	111
data	a	b	c	d	e	f	g	h

3.2 Logisim 시뮬레이션

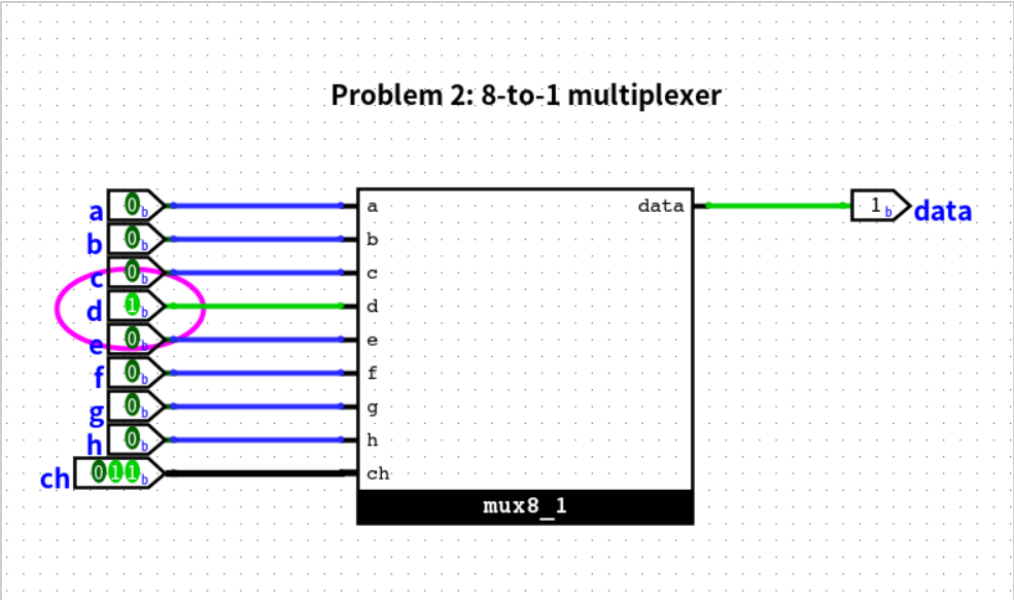


그림 4. ch = 011, d = 1 → data = 1 (d가 선택됨)

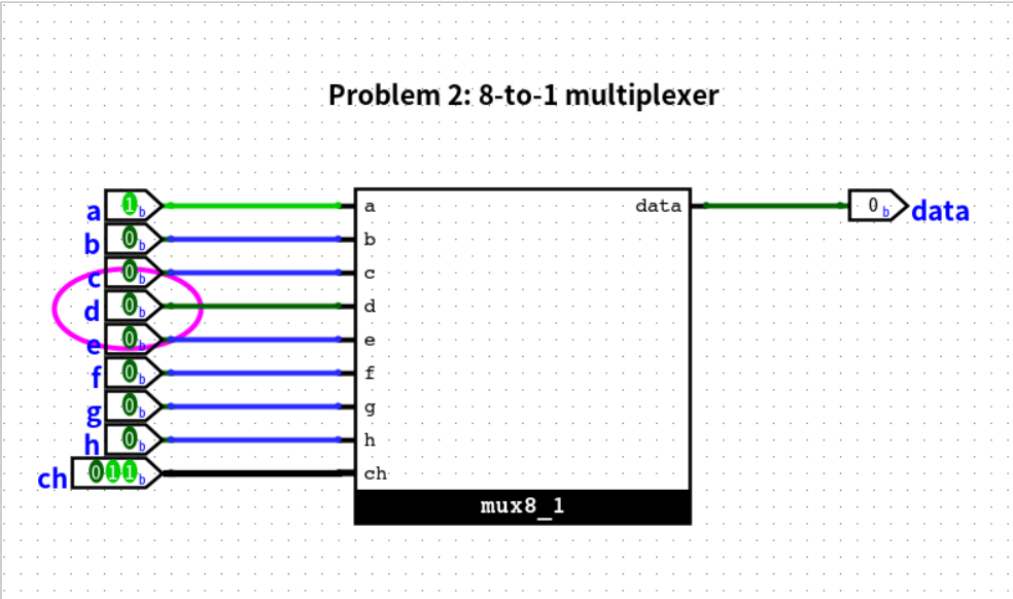


그림 5. ch = 011, a = 1, d = 0 → data = 0 (a는 선택되지 않음)

ch	입력	기대 data	Logisim data	판정
011	d = 1, 나머지 0	1	1	일치
011	a = 1, 나머지 0	0	0	일치

Questa 테스트벤치에서는 ch 8가지 각각에 대해 입력 하나만 1로 바꾸어 가며(64가지) 선택된 입력만 출력되는 것을 확인하였다.

```
# ch=000 selects input 0 OK
# ch=001 selects input 1 OK
# ch=010 selects input 2 OK
# ch=011 selects input 3 OK
# ch=100 selects input 4 OK
# ch=101 selects input 5 OK
# ch=110 selects input 6 OK
# ch=111 selects input 7 OK
# Errors: 0, Warnings: 0
```

4. Problem 3 — 7세그먼트 문자 표시 (a, b, c, d, f, g)

4.1 설계

7세그먼트로 표현할 수 있도록 대문자 A, C, F와 소문자 b, d, g를 사용하였다. 3비트 입력 `dat`로 문자를 고르고, 강의 예제와 같이 `en` = '0'이면 출력을 고임피던스(`"ZZZZZZZZ"`)로 둔다.

```
-- Problem 3: show letters A, b, C, d, F, g on a 7-segment display
-- display(7 downto 0) = a b c d e f g dp
library ieee;
use ieee.std_logic_1164.all;

entity led_letter is
  port (
    en      : in  std_logic;
    dat     : in  std_logic_vector(2 downto 0);
    display : out std_logic_vector(7 downto 0)
  );
end led_letter;

architecture bhv of led_letter is
begin
  process (en, dat)
  begin
    if en = '0' then
      display <= "ZZZZZZZZ";
    else
      case dat is
        when "000"  => display <= "11101110";  -- A
        when "001"  => display <= "00111110";  -- b
        when "010"  => display <= "10011100";  -- C
        when "011"  => display <= "01111010";  -- d
        when "100"  => display <= "10001110";  -- F
        when "101"  => display <= "11110110";  -- g
        when others => display <= "00000000";
      end case;
    end if;
  end process;
end bhv;
```

dat	문자	display (a b c d e f g dp)	켜지는 세그먼트
000	A	11101110	a b c e f g
001	b	00111110	c d e f g
010	C	10011100	a d e f
011	d	01111010	b c d e g
100	F	10001110	a e f g
101	g	11110110	a b c d f g
110, 111	-	00000000	없음
en = 0	-	ZZZZZZZZ	없음 (Hi-Z)

- **if en = '0'**: en이 0이면 display를 구동하지 않는다(Hi-Z). en이 1일 때만 case문으로 문자 패턴을 출력한다.
- Logisim 회로에서는 display 버스를 Splitter로 8개 선으로 나누고, 터널(a~g, dp)로 7-Segment Display의 같은 이름 핀에 연결하였다.

4.2 Logisim 시뮬레이션

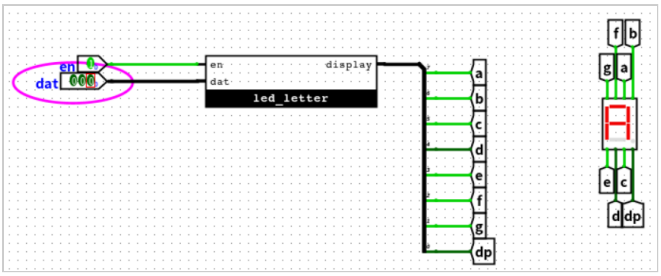


그림 6. en=1, dat=000 → A

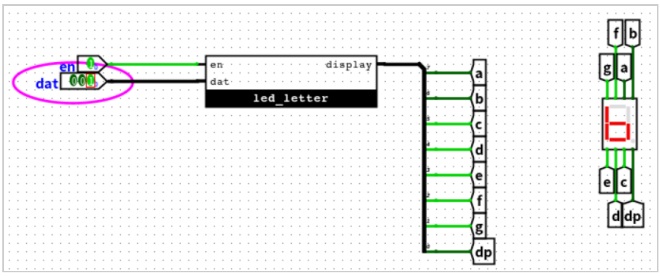


그림 7. en=1, dat=001 → b

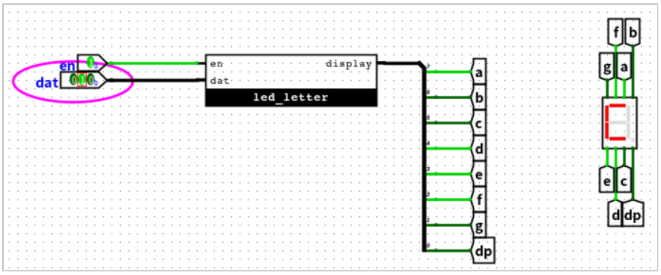


그림 8. en=1, dat=010 → C

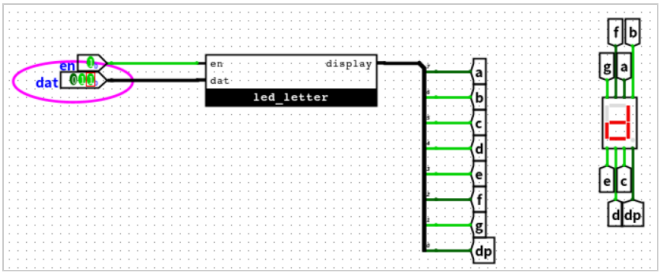


그림 9. en=1, dat=011 → d

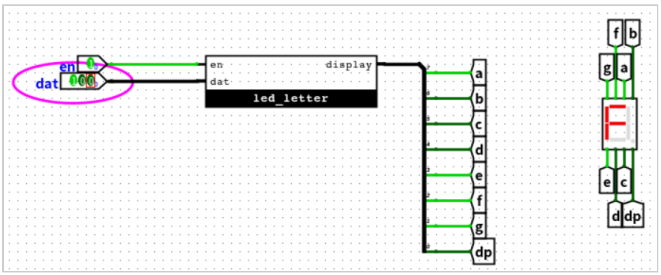


그림 10. en=1, dat=100 → F

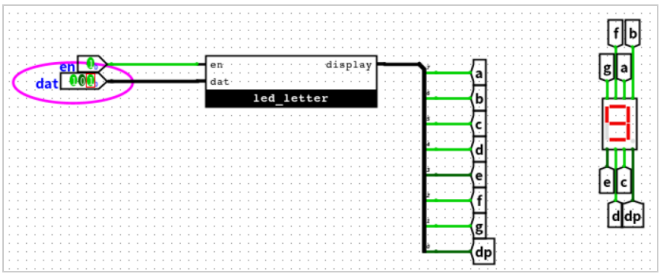


그림 11. en=1, dat=101 → g

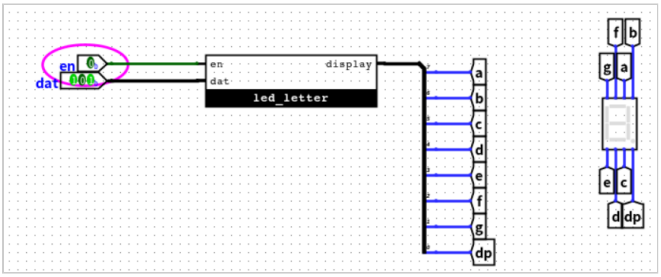


그림 12. en=0 → 꺼짐 (Hi-Z, 파란 선)

여섯 문자가 모두 표에 정리한 모양대로 표시되었다. en = 0일 때는 출력선이 파란색(값이 구동되지 않는 상태)으로 바뀌고 표시기가 꺼졌다.

```
# en=1 dat=000 -> 11101110 (A)
# en=1 dat=001 -> 00111110 (b)
# en=1 dat=010 -> 10011100 (C)
# en=1 dat=011 -> 01111010 (d)
# en=1 dat=100 -> 10001110 (F)
# en=1 dat=101 -> 11110110 (g)
# en=1 dat=110 -> 00000000 (-)
# en=1 dat=111 -> 00000000 (-)
# en=0 -> ZZZZZZZZ
# Errors: 0, Warnings: 0
```

5. Problem 4 — 25인 투표기

5.1 설계

강의의 13인 투표기(`vote13`)와 같은 방식으로, 입력 25개 중 1의 개수를 세어 과반수(13 이상)이면 result = '1'을 출력한다.

```
-- Problem 4: 25-person voting machine
-- result = '1' when 13 or more of the 25 inputs are '1'
library ieee;
use ieee.std_logic_1164.all;

entity vote25 is
  port (
    vt     : in  std_logic_vector(25 downto 1);
    result : out std_logic
  );
end vote25;

architecture bhv of vote25 is
begin
  process (vt)
    variable sum : integer range 0 to 25;
  begin
    sum := 0;
    for i in 1 to 25 loop
      if vt(i) = '1' then
        sum := sum + 1;
      end if;
    end loop;

    if sum >= 13 then
      result <= '1';
    else
      result <= '0';
    end if;
  end process;
end bhv;
```

- **vt : std_logic_vector(25 downto 1)**: 투표자 25명의 입력을 1번부터 25번까지 번호를 붙인 벡터로 받는다.
- **variable sum**: process 안에서 즉시 갱신되는 변수로 1의 개수를 센다. process가 실행될 때마다 0으로 초기화한다.
- **for i in 1 to 25 loop**: 모든 입력을 확인하여 '1'이면 sum을 1 증가시킨다.
- 강의 예제는 루프 안에서 매번 result를 판정했지만, 여기서는 루프가 끝난 뒤 한 번만 `sum >= 13`을 판정하도록 정리하였다. 결과는 같다.
- Logisim 회로에서는 25개짜리 DIP 스위치를 Splitter로 25비트 버스로 묶어 vt에 연결하였다(스위치 k → vt(k)).

5.2 Logisim 시뮬레이션

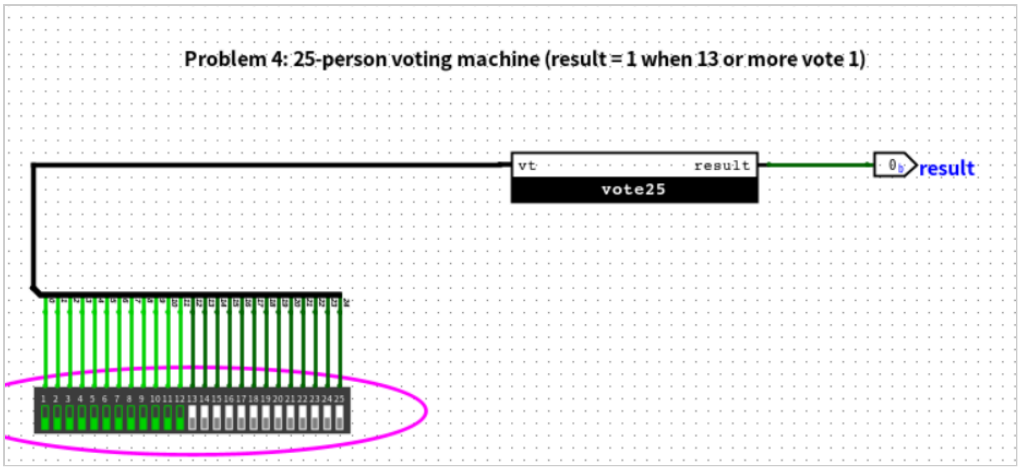


그림 13. 스위치 1~12 ON (12표) → result = 0

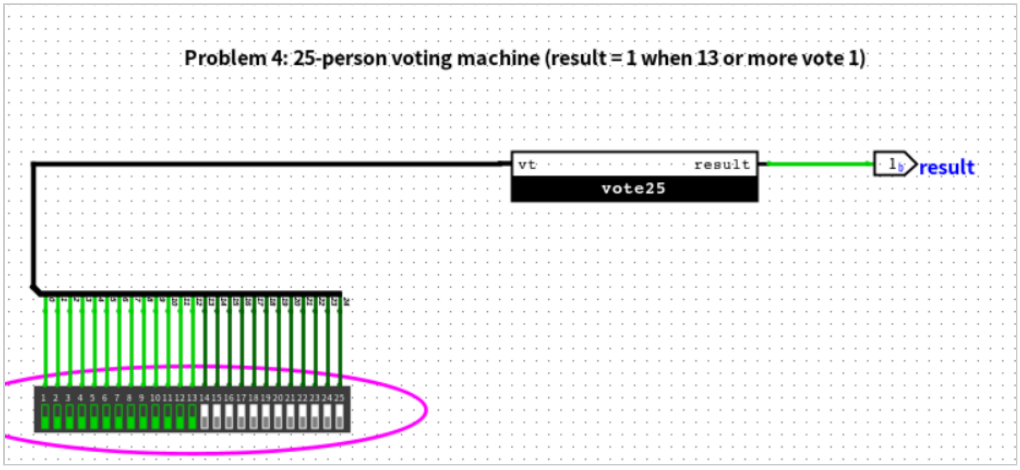


그림 14. 스위치 1~13 ON (13표) → result = 1

찬성 수	기대 result	Logisim result	판정
12	0	0	일치
13	1	1	일치

Questa 테스트벤치에서는 찬성 수 0~25를 모두 넣고, 홀수 번 스위치만 켠 13표/12표 패턴도 확인하였다.

```
# votes=0 -> result='0'
# votes=12 -> result='0'
# votes=13 -> result='1'
# votes=25 -> result='1'
# scattered 13 votes -> result='1'
# scattered 12 votes -> result='0'
# Errors: 0, Warnings: 0
```

6. Problem 5 — 1의 개수를 7세그먼트에 표시

6.1 설계

13비트 입력 `vt`의 '1' 개수를 세어 0~9는 숫자로, 10~15는 A b C d E F로 표시한다. `en` = '0'이면 표시를 끈다.

```
-- Problem 5: count the '1's in a 13-bit vector and show it on a 7-segment display
-- 0-9 -> digits, 10-15 -> A b C d E F, en = '0' -> display off
-- display(7 downto 0) = a b c d e f g dp
library ieee;
use ieee.std_logic_1164.all;

entity count_ones is
  port (
    en      : in  std_logic;
    vt      : in  std_logic_vector(13 downto 1);
    display : out std_logic_vector(7 downto 0)
  );
end count_ones;

architecture bhv of count_ones is
begin
  process (en, vt)
    variable sum : integer range 0 to 15;
  begin
    sum := 0;
    for i in 1 to 13 loop
      if vt(i) = '1' then
        sum := sum + 1;
      end if;
    end loop;

    if en = '0' then
      display <= "00000000";
    else
      case sum is
        when 0      => display <= "11111100";
        when 1      => display <= "01100000";
        when 2      => display <= "11011010";
        when 3      => display <= "11110010";
        when 4      => display <= "01100110";
        when 5      => display <= "10110110";
        when 6      => display <= "10111110";
        when 7      => display <= "11100000";
        when 8      => display <= "11111110";
        when 9      => display <= "11100110";
        when 10     => display <= "11101110";  -- A
        when 11     => display <= "00111110";  -- b
        when 12     => display <= "10011100";  -- C
        when 13     => display <= "01111010";  -- d
        when 14     => display <= "10011110";  -- E
        when others => display <= "10001110";  -- F
      end case;
    end if;
  end process;
end bhv;
```

- **for loop**: Problem 4와 같은 방법으로 1의 개수(sum)를 센다. 입력이 13비트이므로 sum은 0~13이고, 표시 범위(0~15)에 맞춰 `integer range 0 to 15`로 선언하였다.

- **case sum is**: 0~9는 강의자료의 숫자 표, 10~15는 문자 패턴을 출력한다. 13비트 입력으로는 14, 15가 나올 수 없지만 요구사항에 맞춰 E, F도 넣었다.
- **en = '0'**: 문제에서 "display should turn off"라고 하였으므로 모든 세그먼트를 끄는 `"00000000"`을 출력한다.

1의 개수	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
표시	0	1	2	3	4	5	6	7	8	9	A	b	C	d	E	F

6.2 Logisim 시뮬레이션

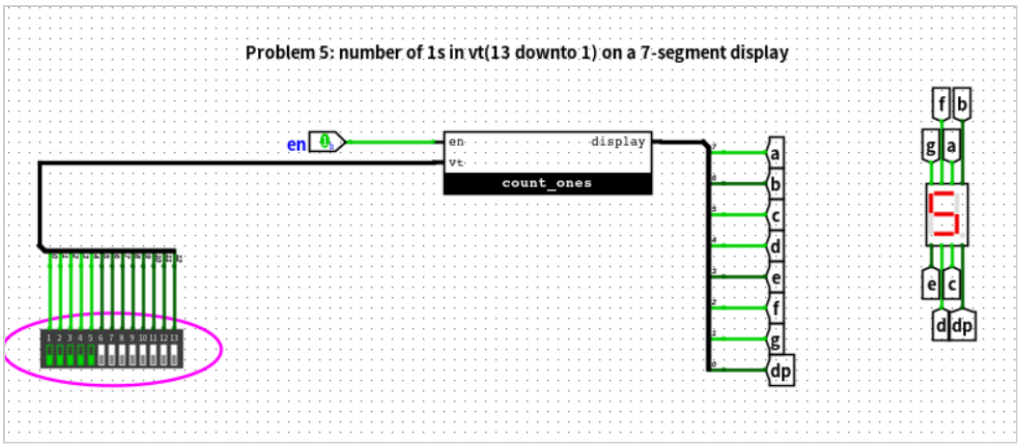


그림 15. en = 1, 스위치 5개 ON → 5

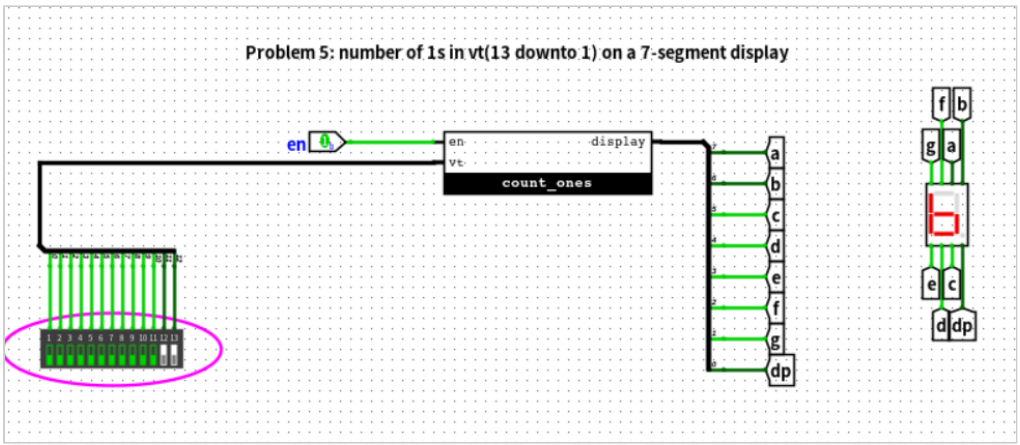


그림 16. en = 1, 스위치 11개 ON → b (11)

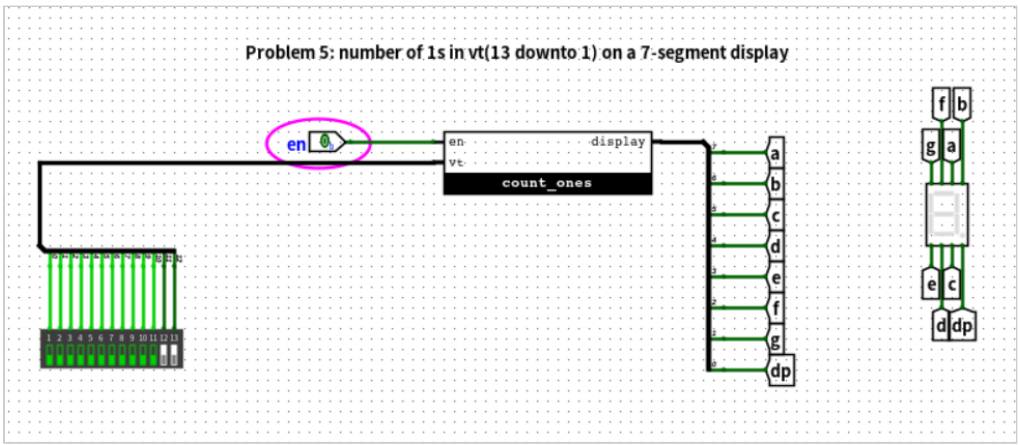


그림 17. en = 0 → 표시 꺼짐

en	1의 개수	기대 표시	Logisim 표시	판정
1	5	5	5	일치
1	11	b	b	일치
0	11	꺼짐	꺼짐	일치

Questa 테스트벤치에서는 1의 개수 0~13과 en = 0을 모두 확인하였다.

```
# ones=0 -> 11111100 (0)
# ones=1 -> 01100000 (1)
# ones=2 -> 11011010 (2)
# ones=3 -> 11110010 (3)
# ones=4 -> 01100110 (4)
# ones=5 -> 10110110 (5)
# ones=6 -> 10111110 (6)
# ones=7 -> 11100000 (7)
# ones=8 -> 11111110 (8)
# ones=9 -> 11100110 (9)
# ones=10 -> 11101110 (A)
# ones=11 -> 00111110 (b)
# ones=12 -> 10011100 (C)
# ones=13 -> 01111010 (d)
# en=0 -> 00000000
# Errors: 0, Warnings: 0
```

7. 결론

- 다섯 문제를 모두 VHDL로 작성하고, Questa 테스트벤치(assert 사용)와 Logisim-evolution 시뮬레이션에서 모두 기대한 결과를 얻었다. 테스트벤치 5개 모두 `Errors: 0, Warnings: 0`이었다.
- 동시 신호 대입문(Problem 1), process와 case문(Problem 2, 3), variable과 for loop를 이용한 개수 세기(Problem 4, 5)를 사용해 보았다.
- 7세그먼트 출력에서 en = 0일 때 Hi-Z("ZZZZZZZZ")로 두는 방법(Problem 3)과 모든 세그먼트를 0으로 끄는 방법(Problem 5)을 비교하였다. Hi-Z는 선을 구동하지 않는 상태로, Logisim에서 파란 선으로 표시되었다.